Python Engineering - Semana 04

Semana 04 - POO, archivos y pruebas

```
01_python_bootcamp/week_04_oop_files_testing/
```

Cómo usar este documento

Este PDF es un índice de repaso de la semana. Cada class resume **qué problema resolvemos, qué conceptos introducimos y qué sintaxis o funciones aparecen**. Si un concepto se te olvida, localízalo aquí y vuelve al ejercicio original.

class_13_classes_objects

EX21 - Clase VerticalState

Programación orientada a objetos: class = definición general; object/instance = realización concreta de esa clase.

Atributos: Datos persistentes del objeto: self.h, self.v, self.t y self.g.

Métodos: Funciones definidas dentro de la clase que operan sobre un objeto concreto.

__init__: Inicializa los atributos cuando se crea una instancia.

self: Referencia al objeto actual; earth_state.specific_energy() pasa earth_state como self.

Estado: euler_step(dt) modifica los atributos del propio objeto; objetos distintos mantienen estados independientes.

Arquitectura: vertical_state.py define la clase; main.py crea earth_state y moon_state y utiliza sus métodos.

class_14_file_io

EX22 - Persistencia y CSV

Entrada/salida de archivos: open(), modos r / w / a y el gestor de contexto with open(...).

Texto: write(), readline(), strip(), split() y conversión mediante float().

Rutas: pathlib.Path, __file__, resolve(), parent y mkdir(exist_ok=True).

CSV: Conversión de números a texto para guardarlos y análisis del texto para reconstruir números.

Validación: Validación de ida y vuelta: datos -> archivo -> datos recuperados; comparación de shape y np.allclose.

Arquitectura: vertical_state.py = modelo; file_io.py = persistencia; main.py = orquestación.

Datos crecientes: list + append si N es desconocido; preasignación de NumPy si N es conocido; np.vstack para unir arrays.

class_15_exceptions_debugging

EX23 - Excepciones y depuración

Objetivo: Responder a fallos previsibles sin ocultarlos.

Excepciones: try, except, else, finally y raise.

Tipos de error: FileNotFoundError y ValueError.

Depuración: puntos de ruptura, panel Variables, pila de llamadas (Call Stack), Step Into / Step Over / Step Out y traceback.

Diseño: Detectar cerca del problema y gestionar donde existe contexto suficiente para decidir.

Aplicación: Carga segura de un CSV válido, inexistente o corrupto.

class_16_testing_project_structure

Pruebas y estructura de proyecto

Objetivo: Cerrar el bootcamp introduciendo verificación automatizada y una estructura mantenible.

Pruebas: Comprobar comportamientos esperados mediante casos reproducibles y comprobaciones automáticas.

Estructura: Separar código, datos, pruebas, configuración y documentación de forma consistente.

Calidad: Evitar regresiones al modificar código y preparar proyectos para portfolio/GitHub.

Resultado: Dejar el bloque preparado para proyectos más grandes y para el salto a métodos numéricos.

Índice rápido de conceptos

Concepto	Dónde verlo
Programación orientada a objetos: class = definición general; object/instance = realización concreta de esa clase.	class_13_classes_objects
Atributos: Datos persistentes del objeto: self.h, self.v, self.t y self.g.	class_13_classes_objects
Métodos: Funciones definidas dentro de la clase que operan sobre un objeto concreto.	class_13_classes_objects
__init__: Inicializa los atributos cuando se crea una instancia.	class_13_classes_objects
self: Referencia al objeto actual; earth_state.specific_energy() pasa earth_state como self.	class_13_classes_objects
Estado: euler_step(dt) modifica los atributos del propio objeto; objetos distintos mantienen estados independientes.	class_13_classes_objects
Arquitectura: vertical_state.py define la clase; main.py crea earth_state y moon_state y utiliza sus métodos.	class_13_classes_objects
Entrada/salida de archivos: open(), modos r / w / a y el gestor de contexto with open(...).	class_14_file_io
Texto: write(), readline(), strip(), split() y conversión mediante float().	class_14_file_io
Rutas: pathlib.Path, __file__, resolve(), parent y mkdir(exist_ok=True).	class_14_file_io
CSV: Conversión de números a texto para guardarlos y análisis del texto para reconstruir números.	class_14_file_io
Validación: Validación de ida y vuelta: datos -> archivo -> datos recuperados; comparación de shape y np.allclose.	class_14_file_io
Arquitectura: vertical_state.py = modelo; file_io.py = persistencia; main.py = orquestación.	class_14_file_io
Datos crecientes: list + append si N es desconocido; preasignación de NumPy si N es conocido; np.vstack para casos intermedios.	class_14_file_io
Objetivo: Responder a fallos previsibles sin ocultarlos.	class_15_exceptions_debugging
Excepciones: try, except, else, finally y raise.	class_15_exceptions_debugging
Tipos de error: FileNotFoundError y ValueError.	class_15_exceptions_debugging
Depuración: puntos de ruptura, panel Variables, pila de llamadas (Call Stack), Step Into / Step Over / Step Out y Continue.	class_15_exceptions_debugging
Diseño: Detectar cerca del problema y gestionar donde existe contexto suficiente para decidir.	class_15_exceptions_debugging

Regla de repaso

Primero busca **el concepto**; después abre el PDF de esa clase; por último ejecuta o modifica el ejercicio. La combinación documento + código es el mecanismo de repaso del curso.